

LOGICAL EXECUTION ORDER

- FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT (aliases, window functions) → DISTINCT → ORDER BY → LIMIT/OFFSET
- SELECT aliases are **not** visible in WHERE/HAVING (they are in ORDER BY).
 - Window functions cannot be in WHERE: compute in a CTE or subquery, filter outside.
 - WHERE filters rows before grouping; HAVING filters groups after aggregation.
 - This is the logical order; the optimiser may execute differently but never changes the result.

QUERY TEMPLATE

```
WITH x AS (SELECT ... FROM t WHERE ...)
SELECT a, COUNT(*) AS n, AVG(v) AS m
FROM x JOIN y ON x.id = y.id
WHERE d ≥ '2025-01-01' AND d < '2025-02-01'
GROUP BY a
HAVING COUNT(*) > 5
ORDER BY n DESC
LIMIT 10;
```

Dates: use half-open ranges ≥ start AND < next, never BETWEEN on timestamps (it misses most of the last day).

JOINS

Join	Keeps
INNER	matches only
LEFT	all left rows; NULLs on the right if no match
RIGHT / FULL	all right / all rows of both
CROSS	every pair (m × n rows)
SELF	table joined to itself (manager, previous row)

- Anti-join** (no match): LEFT JOIN ... WHERE r.id IS NULL, or NOT EXISTS.
- Semi-join** (has a match, no duplication): EXISTS.
- Right-table filter in WHERE after a LEFT JOIN turns it into an inner join: put it in ON.
- Duplicate keys on either side multiply rows (**fan-out**). Check the grain of both tables first.
- USING (id) merges equal-named key columns; NATURAL JOIN is risky, avoid.

NULL RULES

- NULL = NULL is unknown, not true. Use IS NULL / IS NOT NULL.
- COUNT(*) counts rows; COUNT(col) skips NULLs; COUNT(DISTINCT col) unique non-NULLs.
- SUM/AVG/MIN/MAX ignore NULLs: AVG of 1, 2, NULL is 1.5.
- NULL + 1 is NULL. COALESCE(x, 0) = first non-NULL. NULLIF(a, 0) = NULL if equal (safe divide).
- ORDER BY NULL placement differs by engine; use NULLS FIRST/LAST (PG).
- NULL in WHERE a <> 1 is filtered out: NULL rows vanish silently.
- TRAP** NOT IN (subquery) returns nothing when the subquery contains a NULL. Use NOT EXISTS.

AGGREGATION AND SETS

- Aggregates: COUNT SUM AVG MIN MAX, STRING_AGG / GROUP_CONCAT, PERCENTILE_CONT(.5) WITHIN GROUP (ORDER BY x).
- Every non-aggregated SELECT column must be in GROUP BY.
- Conditional counts: SUM(CASE WHEN s = 'x' THEN 1 ELSE 0 END) or COUNT(*) FILTER (WHERE s = 'x') (PG).
- GROUP BY multiple columns; ROLLUP/CUBE for subtotals; GROUPING SETS.
- UNION stacks and removes duplicates; UNION ALL keeps them (faster); INTERSECT, EXCEPT. Same column count and types.
- IN (list) vs EXISTS: EXISTS stops at the first match and is NULL-safe.
- Subquery types: scalar (one value), row/table, correlated (refers to the outer row, runs per row).

- CTE (WITH) = named, readable subquery. Recursive CTE = anchor UNION ALL recursive step.

WINDOW FUNCTIONS

```
fn(x) OVER (
  PARTITION BY g
  ORDER BY d
  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

- Keeps every row (unlike GROUP BY). PARTITION BY = groups; ORDER BY = order within the group.
- Default frame when ORDER BY is present: RANGE UNBOUNDED PRECEDING ... CURRENT ROW. LAST_VALUE needs ... UNBOUNDED FOLLOWING.
- Frames count **rows**, so missing days need a calendar table.

Function	For 90, 70, 70, 50
ROW_NUMBER	1, 2, 3, 4 (ties arbitrary)
RANK	1, 2, 2, 4 (gaps)
DENSE_RANK	1, 2, 2, 3 (no gaps)
NTILE(4)	quartile bucket 1-4

- LAG(x, n) / LEAD(x, n): n rows back / forward. FIRST_VALUE, NTH_VALUE. SUM() OVER (ORDER BY d) = running total. PERCENT_RANK, CUME_DIST for percentiles.
- Window functions run after GROUP BY: SUM(SUM(x)) OVER () is valid.

HANDY FUNCTIONS

- Strings: LOWER TRIM SUBSTR LENGTH CONCAT / ||, LIKE 'a%', REPLACE, SPLIT_PART, REGEXP_REPLACE, POSITION.
- Dates (PG): DATE_TRUNC('month', d), EXTRACT(dow FROM d), d + INTERVAL '7 day', AGE(a, b), CURRENT_DATE.
- MySQL: DATE_FORMAT, DATEDIFF(a, b), DATE_ADD. BigQuery: DATE_DIFF, TIMESTAMP_TRUNC. SQL Server: DATEDIFF(day, a, b), TOP n.
- Cast: CAST(x AS INT), x::numeric (PG). Integer division truncates: write 100.0 * a / b.
- CASE WHEN ... THEN ... ELSE ... END works in SELECT, ORDER BY and inside aggregates.
- Limit rows: LIMIT n OFFSET k (PG/MySQL), FETCH FIRST n ROWS, TOP n.
- DISTINCT ON (k) (PG): first row per k by ORDER BY.

DATE RECIPES (POSTGRESQL)

- Month start: DATE_TRUNC('month', d); next month: + INTERVAL '1 month'.
- Last 7 days: d ≥ CURRENT_DATE - 7.
- Days between: b - a (dates) or EXTRACT(EPOCH FROM b - a) / 86400.
- Weekday (0 = Sunday): EXTRACT(dow FROM d); ISO week: EXTRACT(week FROM d).
- Hour bucket: DATE_TRUNC('hour', ts).
- Time zones: store UTC, convert with ts AT TIME ZONE 'Asia/Kolkata'.
- Calendar table: generate_series(d1, d2, '1 day') fills missing days before a moving average.

INTERVIEW QUESTION -> TOOL

Asked	Reach for
Nth highest, top N per group	DENSE_RANK / ROW_NUMBER in a CTE
Never did X / missing rows	anti-join, NOT EXISTS
Duplicates	GROUP BY ... HAVING COUNT(*) > 1
Change vs previous	LAG
Running or moving total	SUM/AVG OVER + frame
Consecutive days	date - ROW_NUMBER
Retention, cohorts	first-date CTE + LEFT JOIN
Funnel	COUNT(DISTINCT CASE ...)
Rows to columns	SUM(CASE ...) pivot
Hierarchy, org chart	recursive CTE
Median	PERCENTILE_CONT or row-number trick

TOP-N PER GROUP, NTH HIGHEST

```
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (
    PARTITION BY dept ORDER BY sal DESC) r
  FROM emp) t
WHERE r ≤ 3;
-- second highest without windows
SELECT MAX(sal) FROM emp
WHERE sal < (SELECT MAX(sal) FROM emp);
```

Say the tie rule: DENSE_RANK keeps all ties, ROW_NUMBER returns exactly N (add a tie-breaker). Employees paid more than their manager: self-join e.boss = m.id.

DEDUPE, ANTI-JOIN, DUPLICATES

```
-- latest row per user
SELECT * FROM (SELECT *, ROW_NUMBER() OVER
  (PARTITION BY user_id ORDER BY ts DESC) rn
  FROM ev) t WHERE rn = 1;
-- emails that appear twice or more
SELECT email, COUNT(*) FROM u
GROUP BY email HAVING COUNT(*) > 1;
-- users who never ordered
SELECT u.* FROM u WHERE NOT EXISTS
  (SELECT 1 FROM o WHERE o.uid = u.id);
```

Test a DELETE as a SELECT first.

RUNNING TOTAL, MOVING AVERAGE, GROWTH

```
SUM(rev) OVER (ORDER BY d)
AVG(rev) OVER (ORDER BY d ROWS 6 PRECEDING)
rev - LAG(rev) OVER (ORDER BY m)
100.0 * (rev - LAG(rev) OVER (ORDER BY m))
  / NULLIF(LAG(rev) OVER (ORDER BY m), 0)
d - LAG(d) OVER (PARTITION BY uid ORDER BY d)
```

Lines: running total, 7-row average, change, % growth, days between orders. Aggregate to one row per day first, then window. Partition LAG by entity; the first row has NULL.

GAPS AND ISLANDS (STREAKS)

```
-- one row per user per day first (DISTINCT)
SELECT uid, MIN(d) AS start, COUNT(*) AS len
FROM (SELECT uid, d, d - (ROW_NUMBER() OVER
  (PARTITION BY uid ORDER BY d))::int AS grp
  FROM days) t
GROUP BY uid, grp
HAVING COUNT(*) ≥ 3;
```

Consecutive days minus a running counter give a constant: the streak id (PG: date - int is a date). Duplicate days break it.

RETENTION AND COHORTS

```
WITH c AS (SELECT uid, MIN(d) AS d0
  FROM logins GROUP BY uid)
SELECT c.d0, COUNT(DISTINCT c.uid) AS n,
  COUNT(DISTINCT l.uid) * 1.0
  / COUNT(DISTINCT c.uid) AS d7
FROM c LEFT JOIN logins l
  ON l.uid = c.uid AND l.d = c.d0 + 7
GROUP BY c.d0;
```

Cohort = first activity date. Denominator = cohort size, not login rows. LEFT JOIN keeps non-returns. Monthly cohorts: truncate dates to month and join on month offset.

FUNNEL AND SESSIONS

```
SELECT
  COUNT(DISTINCT uid) FILTER (WHERE e='view') v,
  COUNT(DISTINCT uid) FILTER (WHERE e='cart') c,
  COUNT(DISTINCT uid) FILTER (WHERE e='buy') b
FROM ev;
-- flag a new session after a 30 minute gap
WITH g AS (SELECT *, ts - LAG(ts) OVER
  (PARTITION BY uid ORDER BY ts) AS gap FROM ev)
SELECT *, SUM(CASE WHEN gap IS NULL
  OR gap > INTERVAL '30 min' THEN 1 ELSE 0 END)
  OVER (PARTITION BY uid ORDER BY ts) AS sess
FROM g;
```

Count users, not events (portable form: COUNT(DISTINCT CASE WHEN e='buy' THEN uid END)). A strict funnel needs step order: first timestamps must increase view < cart < buy.

PIVOT, SHARE OF TOTAL, MEDIAN

```
SELECT m,
  SUM(CASE WHEN cat='A' THEN rev ELSE 0 END) a,
  SUM(CASE WHEN cat='B' THEN rev ELSE 0 END) b
FROM s GROUP BY m;
100.0 * SUM(rev) / SUM(SUM(rev)) OVER ()
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)
```

Lines: pivot, percent of total, median (PG). Portable median: number rows, take the middle one or average the two middle ones. Pivot columns are fixed in the query.

RECURSIVE CTE, "IN EVERY MONTH"

```
WITH RECURSIVE t AS (
  SELECT id, boss, 1 AS lvl
  FROM emp WHERE boss IS NULL
  UNION ALL
  SELECT e.id, e.boss, t.lvl + 1
  FROM emp e JOIN t ON e.boss = t.id)
SELECT * FROM t;
```

"Did X in all 3 months": GROUP BY uid HAVING COUNT(DISTINCT month) = 3. Churn candidates: last order older than a fixed as-of date. Guard against cycles.

TRAPS CHECKLIST

- **Fan-out**: summing an order-level column after joining items inflates totals. Aggregate before joining.
- **Right-table filter** in WHERE after LEFT JOIN drops unmatched rows. Use ON.
- **Integer division** truncates; BETWEEN on timestamps; COUNT(*) where you need COUNT(DISTINCT user).
- **Ties** in ranking; NOT IN with NULLs; LAST_VALUE default frame; GROUP BY column missing.
- Sanity check after every join: row count and totals reconcile with the source.

SPEED, INDEXES, CONCEPTS

- **Index** = sorted structure (B-tree) for lookups, ranges and joins; costs write speed and space. Composite index: leftmost column first.
- **Sargable**: no function on an indexed column (WHERE YEAR(d) = 2025 is bad, a date range is good). Select only needed columns, filter early, read EXPLAIN.
- DELETE removes rows (rollback ok); TRUNCATE empties fast; DROP removes the table.
- **ACID**: Atomic, Consistent, Isolated, Durable. **OLTP** many small writes; **OLAP** big scans (columnar warehouse).
- Keys: primary (unique, not null), foreign (references). Normalisation cuts redundancy; analytics often uses a star schema.
- Isolation anomalies: dirty read, non-repeatable read, phantom read.
- Clustered index = physical order (one per table); non-clustered = separate pointer structure.
- Large joins: filter and aggregate first; temp table if an expensive result is reused.

LIVE-ROUND ROUTINE

1. Restate the question and the **output grain** (one row per what?).
2. Inspect keys, NULLs, duplicates, date types; ask about ties and cancelled rows.
3. Plan in steps aloud: CTE 1, CTE 2, final select.
4. Write incrementally; check on 2-3 rows by hand.
5. Reconcile row counts after joins; mention indexes and performance.

SAY IT IN 20 SECONDS

- "Before joining, I check the grain of each table."
- "Top N per group: rank in a CTE, filter outside."
- "Retention: cohort by first day, divide by cohort size."
- "WHERE filters rows, HAVING filters groups."
- "I test on two or three rows by hand."